

POSTEK OXScript

API Function Manual

Version: V2.0.0

Copyright

© 2023 by Postek Electronics Co., Ltd. All rights reserved. This manual is protected by copyright laws and may not be reproduced in any form without the prior written permission of POSTEK. No patent liability is assumed regarding the use of the information contained herein.

Trademarks

POSTEK is a registered trademark by Postek Electronics Co., Ltd. In this manual, POSTEK specifically refers to Shenzhen Postek Electronics Co., Ltd.

Other trademarks or companies mentioned in this manual are the property of their respective owners.

Disclaimer

This manual has undergone validation and review for accuracy. The instructions and descriptions provided are accurate for the POSTEK printer at the time of distribution of this manual. However, please note that subsequent printers and manuals may be subject to change without prior notice. POSTEK assumes no liability for any damages, whether direct or indirect, resulting from errors, omissions, or discrepancies between the printer and the information presented in this manual. You can install OX_Script with: `pip3 install ox-script`.

There is an officially supported POSTEK Github page where the source code of all the demo videos and as well as tutorial videos will be posted. The links to the demo videos and tutorial videos can be found on the GitHub page as well.

<https://github.com/POSTEK-OX-Script>

Postek Electronics Co., Ltd.

Address: Wisdom Plaza, Block B, Tower 2, 18th Floor
Qiaoxiang Road, Nanshan District, Shen Zhen,
Guang Dong 518052

Web site: <http://www.postekchina.com>

Tel: +86-755-83240988 400-893-3288

Email: Sales - Overseas@postek.com.cn

Technical Support - Tech@postek.com.cn

Table of Content

1 User Manual	2
1.1 Supported Deveices	2
1.2 Python File Format Description	2
2 Introduction	3
2.1 Dot Size	3
2.2 Printer Coordinate System	3
3 API Function	4
3.1 Unit Setting	4
3.1.1 PTK_SetUnit	4
3.2 Port Operation	5
3.2.1 PTK_GetPortData	5
3.3 Printer Setting	7
3.3.1 PTK_CutPage	7
3.3.2 PTK_PrintConfiguration	7
3.3.3 PTK_MediaCalibration	8
3.3.4 PTK_FeedMedia	8
3.4 Label Setting	10
3.4.1 PTK_SetPrintDirection	10
3.4.2 PTK_SetDarkness	10
3.4.3 PTK_SetLabelHeight	11
3.4.4 PTK_SetLabelWidth	15
3.4.5 PTK_SetCoordinateOrigin	16
3.4.6 PTK_SetPrintSpeed	16
3.4.7 PTK_PrintLabel	17
3.5 Print Text	18
3.5.1 PTK_DrawText	18
3.6 Print Graphic	20
3.6.1 PTK_DrawGraphics	20
3.6.2 PTK_DrawGraphicsFromBytes	20
3.7 Print Line	21
3.7.1 PTK_DrawRectangle	21
3.7.2 PTK_DrawDiagonal	22
3.8 Print 2DBarcode	23
3.8.1 PTK_DrawBar2D_QR	23
3.8.2 PTK_DrawBar2D_HANXIN	24
3.8.3 PTK_DrawBar2D_Pdf417	26
3.8.4 PTK_DrawBar2D_DATAMATRIX	29
3.9 Print 1DBarcode	31
3.9.1 PTK_DrawBarcode	31
3.10 RFID Label	35
3.10.1 PTK_GetPrinterStatus	35
3.10.2 PTK_ReadRFID	35
3.10.3 PTK_WriteRFID	36
3.11 UI Interface Customlization	38
3.11.1 PTK_UIInit	38

3.11.2 PTK_UIButton39

3.11.3 PTK_UIText 39

3.11.4 PTK_UITextBox 40

3.11.5 PTK_UIInput40

3.11.6 PTK_UICheckBox41

3.11.7 PTK_UIList 42

3.11.8 PTK_UIPage43

3.11.9 PTK_UIChangePage 44

3.11.10 PTK_UpdateAllFormVariables 44

Preface

Overview

This manual provides a API library for developing flexible Python programs on POSTEK printers.

Target Readers

This manual is primarily intended for the following audience:

- Software Development Engineers
- Technical Support Personnel

Version

Version 2.0.0, published in June 2023.

1 User Manual

1.1 Supported Deveices

This API library is only supported on POSTEK Printers with a LCD touch screen.

1.2 Python File Format Description

This API library allows customization of the LCD screen interface on the printer, and the script file needs to be operated on the LCD screen, thus the written Python program has certain format requirements:

Before using this API library, import the library named "ox_script" in your Python program by adding the line "from ox_script import *" or "import ox_script" at the beginning.

Example

```
from ox_script import *
```

2 Introduction

2.1 Dot Size

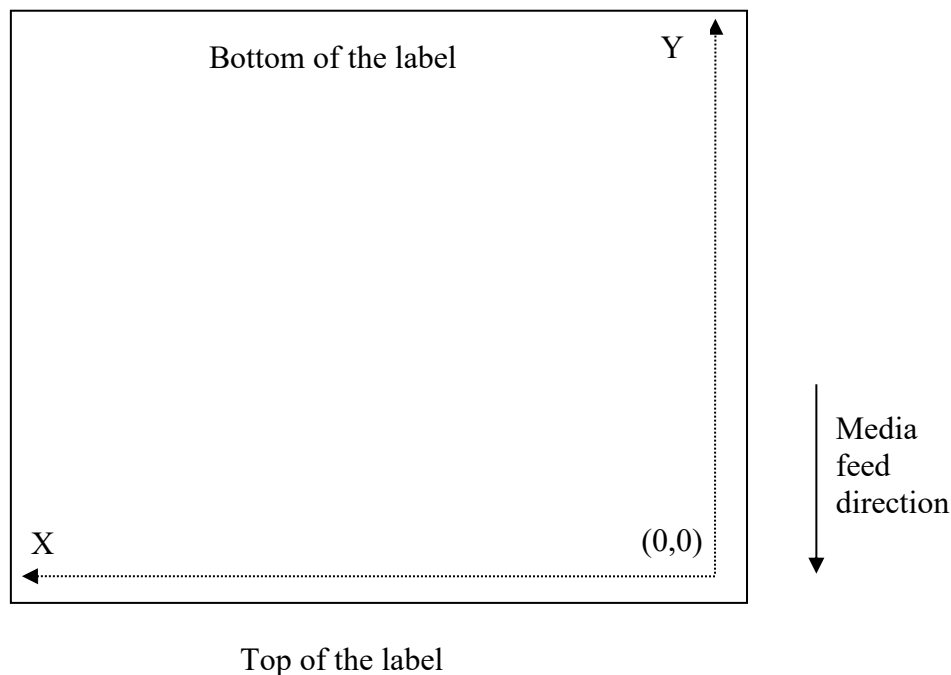
A dot is the smallest unit of printing on a printer. The dot sizes for printers with different resolutions are as follows:

- For 203 DPI printers, 1dot = $25.4\text{mm}/203 \approx 0.125\text{mm}$ (1dot $\approx$ 5mil)
- For 300 DPI printers, 1dot = $25.4\text{mm}/300 \approx 0.085\text{mm}$ (1dot $\approx$ 3mil)
- For 600 DPI printers, 1dot = $25.4\text{mm}/600 \approx 0.042\text{mm}$ (1dot $\approx$ 2mil)

2.2 Printer Coordinate System

The coordinate origin is located at the top left corner of the label, and the coordinate system is defined as shown in the figure below.

It is important to note that due to the downward media feed direction, the label is placed upside down in the figure below. However, for the sake of clarity and consistency, the printout examples in this manual are presented with the label placed top-up.



3 API Function

3.1 Unit Setting

3.1.1 PTK_SetUnit

Description

Set Unit to mm or dots. Default: mm.

Function Definition

```
def PTK_SetUnit(Units):
```

Parameters

Units Set Unit.
 MM - Set Unit to mm.
 DOTS - Set Unit to dots.

Return Value

None

Example

```
PTK_SetUnit(DOTS)
```


3.2 Port Operation

3.2.1 PTK_GetPortData

Description

Get data from specific port of the printer.

Function Definition

```
def PTK_GetPortData(port, dataLen = 4096, timeout = 5000):
```

Parameters

port	Get the sequence number of port type. PORT_SERIAL - The Data Ring Buffer Sequence Number of The Serial Port PORT_USBDEVICE - The Data Ring Buffer Sequence Number Of The Usb Device PORT_ETHERNET - The Data Ring Buffer Sequence Number Of The Ethernet Port PORT_BLUETOOTH - The Data Ring Buffer Sequence Number Of The Bluetooth PORT_WIFI - The Data Ring Buffer Sequence Number Of The WIFI PORT_PARALLEL - The Data Ring Buffer Sequence Number Of The Parallel PORT_WEBSOCK - The Data Ring Buffer Sequence Number Of The WEBSOCK PORT_USBHOST - The Data Ring Buffer Sequence Number Of The USBHOST
dataLen	Length of data.
timeout	Timeout value.

Return Value

string -> data from specify port type
error_string -> Error Info

Example

```
str2 = PTK_GetPortData(PORT_SERIAL)  
print(str2)
```

3.2.2 PTK_WriteToSerial

Description

Write data to serial port of the printer.

Function Definition

```
def PTK_PTK_WriteToSerial (dataBuff):
```

Parameters

dataBuff Specifies the information intended to be written onto Serial.

Example

```
PTK_GetPortData(' %ETAR;')
```

3.2.3 PTK_GetExternalStorageDeviceList**Description**

Get external storage devices name of the printer.

Function Definition

```
def PTK_GetExternalStorageDeviceList():
```

Parameters

None

Return Value

list -> A list file that records all PTK_ExternalStorageDevice instance.

Example

```
storageDeviceList = PTK_GetExternalStorageDeviceList()  
print(storageDeviceList)
```

3.3 Printer Setting

3.3.1 PTK_CutPage

Description

Specifies the number of labels that need to be printed before the printer's cutter performs a cut. The specified number will be saved after power on/off cycle if save_in_flash is true.

Function Definition

```
def PTK_CutPage(number_of_labels, save_in_flash = False):
```

Parameters

number_of_labels	Number of labels printed before cutting. Value range: 0 to 999. Default: 1. When '0' is specified, the cutter will not be enabled.
save_in_flash	Whether the specified number will be saved after a power on/off cycle. Default: The number will not be saved.

Return Value

True	-> OK
error_string	-> Error Info

Example

```
PTK_CutPage(3)#Makes a cut after every three label  
PTK_PrintLabel(number_of_label=6, number_of_copy=1)#print 6 labels
```

3.3.2 PTK_PrintConfiguration

Description

generate a printout containing the current configurations of the printer, commonly referred to as a "self-test page".

Function Definition

```
def PTK_PrintConfiguration():
```

Parameters

None

Return Value

True	-> OK
error_string	-> Error Info

Example

PTK_PrintConfiguration()

3.3.3 PTK_MediaCalibration

Description

Instructs the printer to perform Media Sensor Calibration.

Function Definition

```
def PTK_MediaCalibration():
```

Parameters

None

Return Value

True	-> OK
error_string	-> Error Info

Example

```
PTK_MediaCalibration()
```

3.3.4 PTK_FeedMedia

Description

Feed a blank label.

Function Definition

```
def PTK_FeedMedia():
```

Parameters

None

Return Value

True	-> OK
error_string	-> Error Info

Example

```
PTK_FeedMedia()
```

3.3.5 PTK_SendCmdToPrinter

Description

Send command to Printer directly.

Function Definition

```
def PTK_SendCmdToPrinter (cmd):
```

Parameters

cmd Specifies string or byte data to send to the printer.

Return Value

True -> OK
error_string -> Error Info

Example

```
PTK_SendCmdToPrinter(b"#OXScript;check;0;#End")
```

3.4 Label Setting

3.4.1 PTK_SetPrintDirection

Description

Set the printing orientation of the label. Invalid while not checking using command or drive value on OX_Script. The set data will be loss after a power on/off cycle.

Function Definition

```
def PTK_SetPrintDirection(direction):
```

Parameters

direct Label print direction.
TOP_LEFT- Prints from bottom to top.



BOTTOM_RIGHT- Prints from top to bottom.



Return Value

True -> OK
error_string -> Error Info

Example

```
PTK_SetPrintDirection(BOTTOM_RIGHT)
```

3.4.2 PTK_SetDarkness

Description

Set the print density. Invalid while not checking using command or drive value on OX_Script. The set data will be loss after a power on/off cycle.

Function Definition

```
def PTK_SetDarkness(darkness):
```

Parameters

darkness Selects the density level. The density level can be chosen from different density.
Value range: 0.0 to 30.0

Return Value

True -> OK
error_string -> Error Info

Example

```
PTK_SetDarkness(darkness = 12)
```

3.4.3 PTK_SetLabelHeight

Description

Set the label length and the marker size. The set data will be loss after a power on/off cycle.

Function Definition

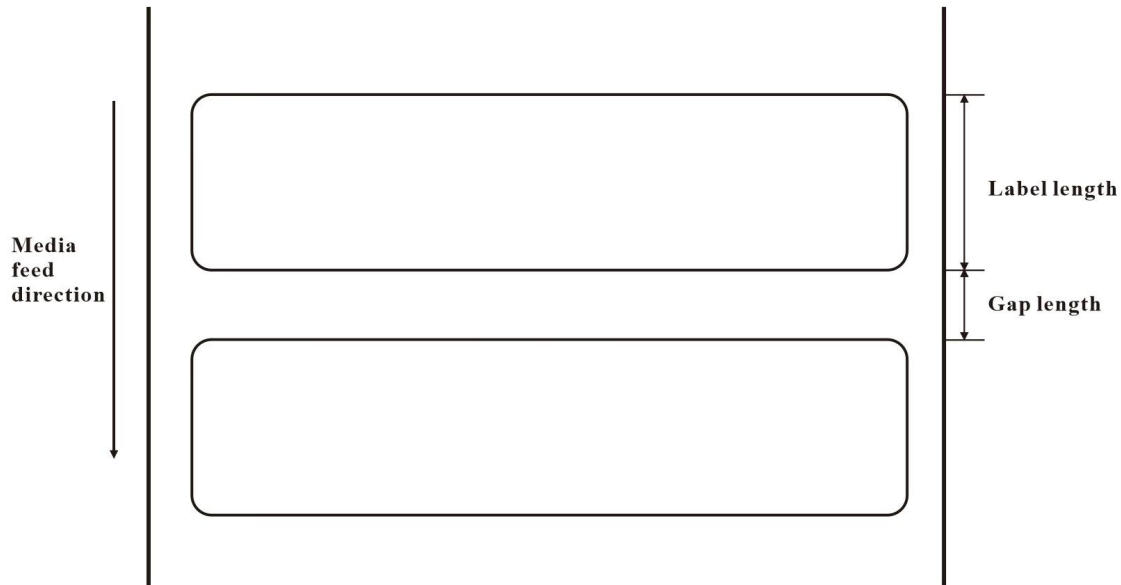
```
def PTK_SetLabelHeight(height, mode= GAP_MODE, gapH = 3):
```

Parameters

height The length of the label. The unit can be set by PTK_SetUnit. Default: mm.
mode Paper gap mode
 GAP_MODE - Gap Mode, Default mode, gapH indicate The distance between the
 intended tear-off line or cutting line and the top of the label. If continuous media is
 used, the maker size is set to 0;
 SPECIAL_MODE - Special Mode, e.g. labels with notch;
 BLACK_LINE_MODE - Black line Mode, gapH is the height of the black line.
gapH The height of the positioning gap/black line/perforation between labels can be set by
 PTK_SetUnit.the value of gapH is related to the label positioning method. Default:
 mm.

Return Value

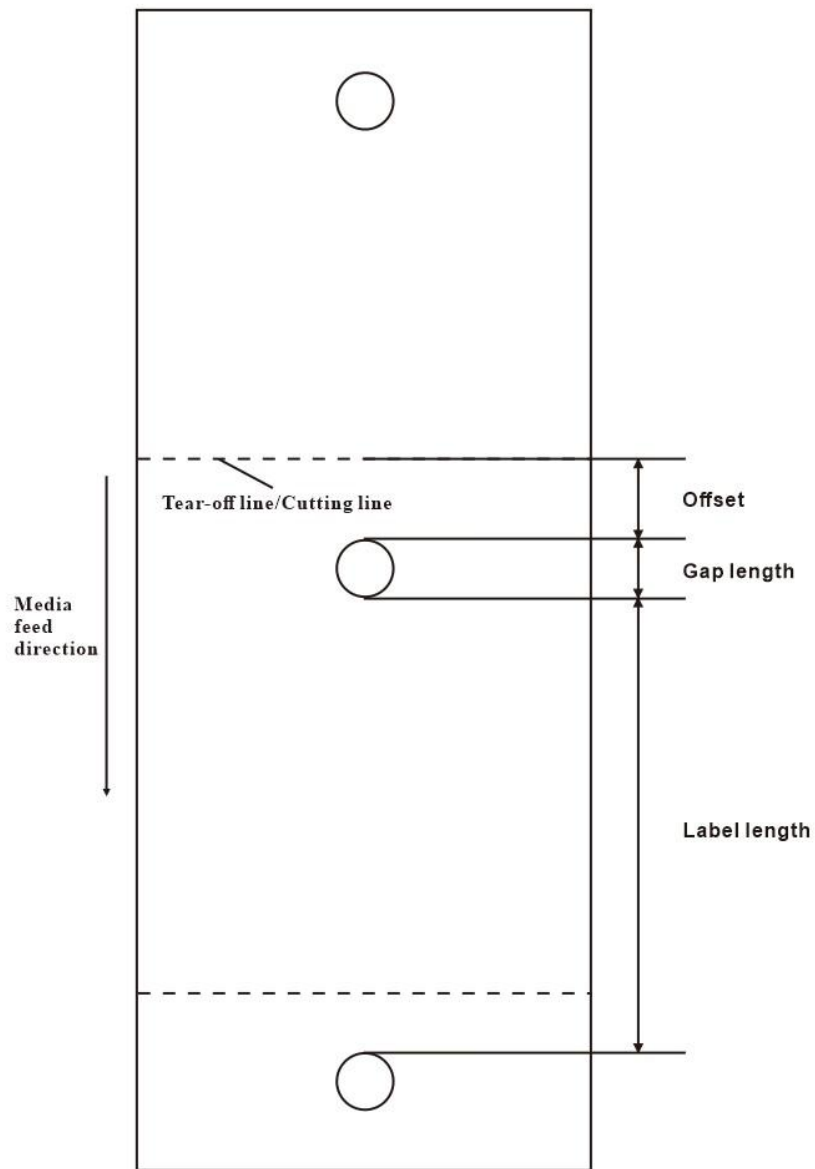
True -> OK
error_string -> Error Info

Example 1: Label with gap marker

label length =20mm

gap length =3mm

PTK_SetLabelHeight(height = 20, mode= GAP_MODE, gapH = 3)

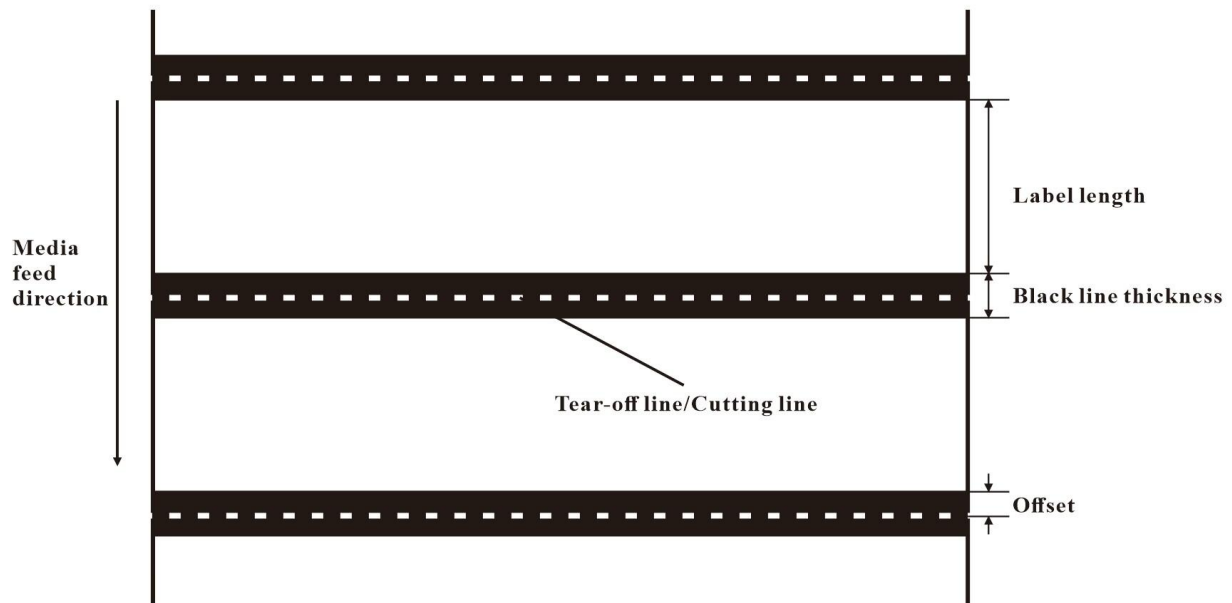
Example 2: Label with notch

label length =12.5mm

gap length =3mm

offset =3mm

PTK_SetLabelHeight(height = 12.5, mode= SPECIAL _MODE, gapH ='3+3')

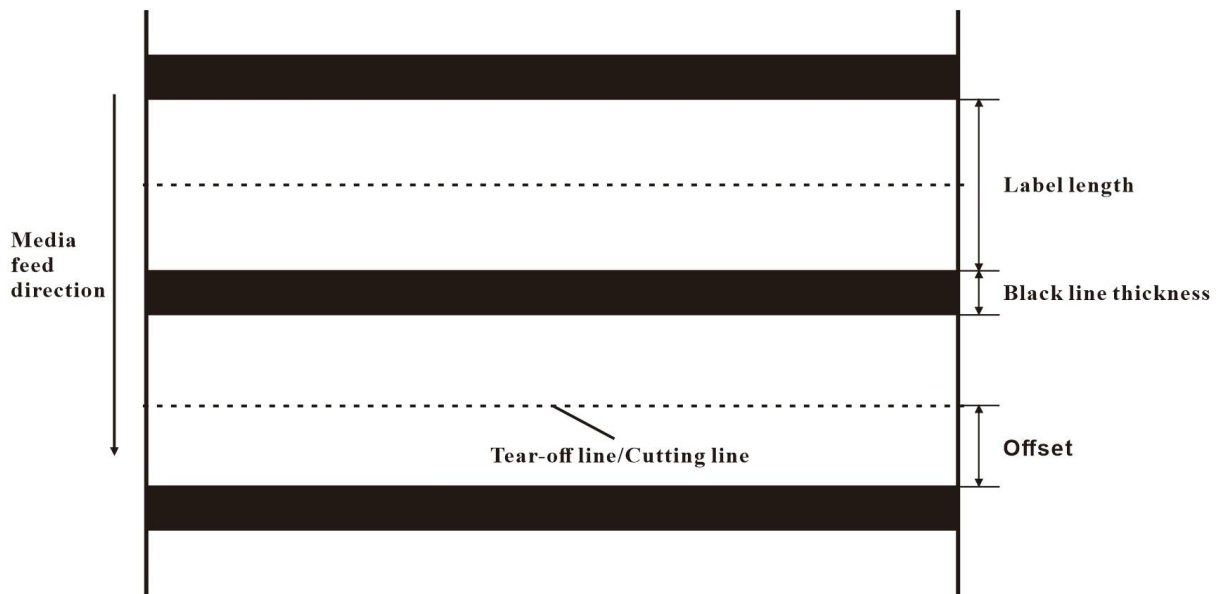
Example 3: Label with black line marker

label length = 31.0mm

gap length = 7mm

offset = 3.5mm

PTK_SetLabelHeight(height = 31, mode= BLACK_LINE_MODE, gapH = '7-4')

Example 4: Label with black line marker

label length = 31 mm
 gap length = 7mm
 offset = 17mm

```
PTK_SetLabelHeight(height = 31, mode= BLACK_LINE_MODE, gapH = '7-17')
```

Note: If this function is not set, the height measured by the label calibration will prevail.

3.4.4 PTK_SetLabelWidth**Description**

Set the width of the label. The set data will be loss after a power on/off cycle.

Function Definition

```
def PTK_SetLabelWidth(width):
```

Parameters

width The width of the label. The unit can be set by PTK_SetUnit. Default: mm.

Return Value

True -> OK
 error_string -> Error Info

Example

```
PTK_SetLabelWidth(width = 1228)  

PTK_PrintLabel(number_of_label = 1, number_of_copy = 1))
```

3.4.5 PTK_SetCoordinateOrigin

Description

Move the coordinate origin to a specified reference point.

Function Definition

```
def PTK_SetCoordinateOrigin(x=0, y=0):
```

Parameters

x The horizontal coordinate of the reference point. Default: 0
y The vertical coordinate of the reference point. Default: 0

Return Value

True -> OK
error_string -> Error Info

Example

```
PTK_SetCoordinateOrigin(x=5,y=5)
```

3.4.6 PTK_SetPrintSpeed

Description

Adjust the print speed of the printer. The set data will be loss after a power on/off cycle.

Function Definition

```
def PTK_SetPrintSpeed(speed):
```

Parameter

speed Specifies the desired print speed. The valid value ranges and their corresponding speeds depend on your printer's firmware format and version. Please refer to the appropriate range based on your printer's firmware.

'speed' value	Print speed
0.5	0.5ips (12.7mm/s)
1	1ips (25.4mm/s)
2	2ips (50.8mm/s)
3	3ips (76.2mm/s)
4	4ips (101.6mm/s)
5	5ips (127.0mm/s)
6	6ips (152.4mm/s)
7	7ips (177.8mm/s)
8	8ips (203.2mm/s)
9	9ips (228.6mm/s)
10	10ips (254.0mm/s)
11	11ips (279.4mm/s)
12	12ips (304.8mm/s)

'speed' value	Print speed
13	13ips (330.2mm/s)
14	14ips (355.6mm/s)
15	15ips (381.0mm/s)
16	16ips (406.4mm/s)
17	17ips (431.8mm/s)
18	18ips (457.2mm/s)
19	19ips (482.6mm/s)
20	20ips (508.0mm/s)

Return Value

True -> OK
error_string -> Error Info

Example

```
PTK_SetPrintSpeed(3)
PTK_PrintLabel(number_of_label = 1, number_of_copy = 1)
```

3.4.7 PTK_PrintLabel**Description**

Set the number of labels to print and number of copies to print.

Function Definition

```
def PTK_PrintLabel(number_of_label, number_of_copy):
```

Parameters

number_of_label Number of labels to print.
Range: 1 ~ 65535.
number_of_copy Number of copies of each label.
Range: 1 ~ 65535.

Return Value

True -> Print Success
error_string -> Error Info

Example

```
PTK_SetUnit(DOTS)
PTK_SetLabelHeight(height=200, gapH=0) # Set the label height to 200dots continuous paper
PTK_PrintLabel(number_of_label=3, number_of_copy=2)# Start printing labels, print 3 sheets,
each copy 2 times
```

Note: After setting the printing content, you need to call this function to start printing.

3.5 Print Text

3.5.1 PTK_DrawText

Description

Prints fixed text, variables, or a set of serial numbers.

Function Definition

```
def PTK_DrawText(x_coordinate,
                 y_coordinate,
                 data,
                 fonts="1",
                 font_size=3,
                 text_style= NORMAL,
                 rotation= 0,
                 ):

```

Parameters

- x_coordinate** The horizontal coordinate of the start point of the text. The unit can be set by PTK_SetUnit. Default: mm.
- y_coordinate** The vertical coordinate of the start point of the text. The unit can be set by PTK_SetUnit. Default: mm.
- rotation** Select rotation.
 0 - no rotation
 1 - rotate 90 degrees
 2 - rotate 180 degrees
 3 - rotate 270 degrees
- fonts** Font selection.
 1 - 5 - built-in Latin fonts
 6 - built-in 24*24 dot matrix Chinese fonts
 7 and 8 - built-in scalable fonts
 A - Z - Downloaded Truetype fonts

Dot matrix fonts:

ID	203DPI	300DPI/600DPI	Note
1	8 x 12dot, Character space 2dots	12 x 20dots, Character space 2dots	Unicode 0~255 printable characters
2	10 x 16dots, Character space 2dots	16 x 28dots, Character space 2dots	Unicode 0~255 printable characters
3	12 x 20dots, Character space 2dots	20 x 36dots, Character space 3dots	Unicode 0~255 printable characters
4	14 x 24dots, Character space 2dots	24 x 44dots, Character space 3dots	Unicode 0~255 printable characters
5	32 x 48dots, Character space	48 x 80dots, Character space	Unicode 0~255 printable characters

ID	203DPI	300DPI/600DPI	Note
	3dots	3dots	No lowercase characters
6	Reserved		

Scalable fonts:

ID	Character Support
7(8MB FLASH)	GB2312 Chinese character set
7(64MB FLASH)	GBK Chinese character set
8	Covers characters from Basic Latin, Latin1, Latin Extended A, and Cyrillic encoding tables.
Font Filename with Extension	Font installed through POSTEK APP

- font_size** When fonts is set to built-in font (1 ~ 6), at this time font_size to set the dot matrix magnification factor, you can select: 1 ~ 24, the default is 3. When fonts select TrueType font (A~Z), The size of the font is determined by the font_size parameter, and the unit can be set by PTK_SetUnit, and the default is mm (unlimited size).
- text_style** Reverse printing.
N - normal printing
R - reverse printing
- data** To include variables in the text data, it is recommended to take advantage of f-strings that come with Python 3.6+. F-strings can provide a fast and easy way to integrate numeric variables, string variables, container variables, date-times, or your own custom objects as long as __str__ or __repr__ method is defined for the class. The combined data string cannot exceed a length of 100 characters.

Return Value

True → OK
error_string → Error Info

Example

```
PTK_DrawText(x_coordinate=20,
             y_coordinate=20,
             fonts="l",
             font_size=3,
             data='156894',
             text_style= NORMAL,
             rotation= 0
             )
```

```
PTK_PrintLabel(number_of_label=1, number_of_copy=1)#start print label
```

Note: After setting this function, you need to call PTK_PrintLabel to start printing.

3.6 Print Graphic

3.6.1 PTK_DrawGraphics

Description

Print a graph stored in the memory of the printer.

Function Definition

```
def PTK_DrawGraphics(px,py,pcxname):
```

Parameters

px The horizontal coordinate of the start point of the graph. The unit can be set by PTK_SetUnit. Default: mm.
py The vertical coordinate of the start point of the graph. The unit can be set by PTK_SetUnit. Default: mm.
pcxname The name of the selected graph to be printed, with a maximum length of 16 characters.

Return Value

True -> OK
error_string -> Error Info

Example

```
PTK_DrawGraphics (px=0, py=500, pcxname="P1")  
PTK_PrintLabel(number_of_label=1, number_of_copy=1)
```

Note: After setting this function, you need to call PTK_PrintLabel to start printing.

3.6.2 PTK_DrawGraphicsFromBytes

Description

Set to prints a binary graph directly without storing it to the printer's memory.

Function Definition

```
def PTK_DrawGraphicsFromBytes(x, y, isbase64, graphic_data):
```

Parameters

x The horizontal coordinate of the start point of the graph. The unit can be set by PTK_SetUnit. Default: mm.
y The vertical coordinate of the start point of the graph. The unit can be set by PTK_SetUnit. Default: mm.
isbase64 Is base64 encode data.
graphic_data Image byte data.

Return Value

True -> OK
False -> The graphic_data passed in is bytes

Example

```
data=b'your graphic bytes data'  
PTK_DrawGraphicsFromBytes (x=0, y=0, isbase64=True, graphic_data=data)
```

3.7 Print Line

3.7.1 PTK_DrawRectangle

Description

Draw a rectangular shape.

Function Definition

```
def PTK_DrawRectangle(x_coordinate,  
                      y_coordinate,  
                      thickness,  
                      end_point_x,  
                      end_point_y):
```

Parameters

x_coordinate	The horizontal coordinate of the start point of the rectangle. The unit can be set by PTK_SetUnit. Default: mm.
y_coordinate	The vertical coordinate of the start point of the rectangle. The unit can be set by PTK_SetUnit. Default: mm.
thickness	The thickness of the border lines. The unit can be set by PTK_SetUnit. Default: mm.
end_point_x	The horizontal coordinate of the end point of the rectangle. The unit can be set by PTK_SetUnit. Default: mm.
end_point_y	The vertical coordinate of the end point of the rectangle. The unit can be set by PTK_SetUnit. Default: mm.

Return Value

True -> OK
error_string -> Error Info

Example

```
PTK_DrawRectangle(x_coordinate = 10, y_coordinate = 10, thickness = 1, end_point_x = 50,  
end_point_y = 50)  
PTK_PrintLabel(number_of_label=1, number_of_copy=1)
```

Note: After setting this function, you need to call PTK_PrintLabel to start printing.

3.7.2 PTK_DrawDiagonal

Description

Draw a diagonal line.

Function Definition

```
def PTK_DrawDiagonal(x_coordinate,  
                     y_coordinate,  
                     thickness,  
                     end_point_x,  
                     end_point_y):
```

Parameters

x_coordinate	The horizontal coordinate of the start point of the line. The unit can be set by PTK_SetUnit. Default: mm.
y_coordinate	The horizontal coordinate of the start point of the line. The unit can be set by PTK_SetUnit. Default: mm.
thickness	The thickness of the border lines. The unit can be set by PTK_SetUnit. Default: mm.
end_point_x	The horizontal coordinate of the end point of the line. The unit can be set by PTK_SetUnit. Default: mm.
end_point_y	The vertical coordinate of the end point of the line. The unit can be set by PTK_SetUnit. Default: mm.

Return Value

True	-> OK
error_string	-> Error Info

Example

```
PTK_DrawDiagonal(x_coordinate = 10, y_coordinate = 10, thickness = 2, end_point_x = 50,  
end_point_y = 50)  
PTK_PrintLabel(number_of_label=1, number_of_copy=1)
```

Note: After setting this function, you need to call PTK_PrintLabel to start printing.

3.8 Print 2DBarcode

3.8.1 PTK_DrawBar2D_QR

Description

Prints QR barcodes.

Function Definition

```
def PTK_DrawBar2D_QR(
    x_coordinate,
    y_coordinate,
    qr_version=0,
    rotation=0,
    multiplier=1,
    encode_mode = 4,
    correction_level=0,
    masking=8,
    data='string'):
```

Parameters

x_coordinate The horizontal coordinate of the start point of the barcode. The unit can be set by PTK_SetUnit. Default: mm.

y_coordinate The horizontal coordinate of the start point of the barcode. The unit can be set by PTK_SetUnit. Default: mm.

qr_version Symbol size. The amount of modules in a barcode. The smallest possible version of a QR Code comprises 21×21 modules. The more data to be encoded, the more modules need to be added. QR Codes grow in steps of four modules in each direction up to a maximum of 177×177 modules (version 40). The values of the 'symbol size' and their corresponding amount of modules are as follows.

Value	Amount of Modules
0	auto match
1	21×21
2	25×25
3	29×29
...	
40	177×177

rotation Select rotation.

0 - no rotation

1 - rotate 90 degrees

2 - rotate 180 degrees

3 - rotate 270 degrees

multiplier Module size. The size, of a module in both directions. Range: 1 to 99

encode_mode QR code encoding mode.

0 - Numeric mode

1 - Alphanumeric mode

2 - Byte mode 0 ~ 256

3 - Chinese characters mode

4 - Mixed mode

correction_level The error correction level. QR codes have four correction levels. A higher level can be used for a warehouse where the code is more likely to get dirty. The higher the EC level, the higher the ability to correct errors, but the larger the code will be.

Value	Amount of Modules
1	Level L (low) 7% of code words can be restored
2	Level M (medium) 15%
3 (default)	Level Q (quartile) 25%
4	Level H (high) 30%

masking QR code mask patterns.

- 0 - Mask pattern 000
- 1 - Mask pattern 001
- 2 - Mask pattern 010
- 3 - Mask pattern 011
- 4 - Mask pattern 100
- 5 - Mask pattern 101
- 6 - Mask pattern 110
- 7 - Mask pattern 111
- 8 - Auto Selection

data The QR barcode data.

Return Value

True -> OK
 error_string -> Error Info

Example

```
PTK_DrawBar2D_QR(
    x_coordinate=200,
    y_coordinate=200,
    multiplier=5,
    data=' ABCabc12345',
    encode_mode=2)
```

Printout



Note: After setting this function, you need to call PTK_PrintLabel to start printing.

3.8.2 PTK_DrawBar2D_HANXIN

Description

Print HANXIN Barcode

Function Definition

```
def PTK_DrawBar2D_HANXIN(
    x_coordinate,
    y_coordinate,
    multiplier,
    data,
    encoding= 0,
    correction_level= 0,
    masking= 0,
    rotation= 0):
```

Parameters

x_coordinate	The horizontal coordinate of the start point of the barcode. The unit can be set by PTK_SetUnit. Default: mm.
y_coordinate	The horizontal coordinate of the start point of the barcode. The unit can be set by PTK_SetUnit. Default: mm.
rotation	Select rotation. 0 - no rotation 1 - rotate 90 degrees 2 - rotate 180 degrees 3 - rotate 270 degrees
multiplier	Module size. The size, of a module in both directions. Range: 1 to 30.
encoding	Encoding mode. 0 - Numeric mode 1 - Text mode 2 - Binary mode 3 - Commonly Used Chinese Characters Zone 1 Mode Encoding 4 - Commonly Used Chinese Characters Zone 2 Mode Encoding 5 - GB 18030 Double-Byte Mode Encoding 6 - GB 18030 Four-Byte Mode Encoding
correction_level	Error correction level. 0 - Level 0 1 - Level 1 2 - Level 2 3 - Level 3
masking	Mask patterns. 0 - Mask pattern 000 1 - Mask pattern 01 2 - Mask pattern 10 3 - Mask pattern 11
data	The HANXIN barcode data.

Return Value

True -> OK
error_string -> Error Info

Example

```
PTK_DrawBar2D_HANXIN(
    x_coordinate= 200,
```

```

y_coordinate= 100,
multiplier= 3,
data= "1651651",
encoding= 1,
correction_level= 3,
masking= 2,
rotation= 0)

```

```
PTK_PrintLabel(number_of_label=1, number_of_copy=1)
```

Printout



Note: After setting this function, you need to call PTK_PrintLabel to start printing.

3.8.3 PTK_DrawBar2D_Pdf417

Description

Print PDF417 Barcode.

Function Definition

```

def PTK_DrawBar2D_Pdf417(
    x_coordinate: float,
    y_coordinate: float,
    correction_level:int,
    px:int,
    py:int,
    encode_row:int,
    encode_column:int,
    t:int,
    data:string,
    rotation= 0):

```

Parameters

x_coordinate	The horizontal coordinate of the start point of the barcode. The unit can be set by PTK_SetUnit. Default: mm.
y_coordinate	The horizontal coordinate of the start point of the barcode. The unit can be set by PTK_SetUnit. Default: mm.
correction_level	Error correction level. Range: 0-8
px	Module width. Range: 2 to 9
py	Module height. Range: 4 to 99
encode_row	Maximum number of rows of modules. Range: 3-90
encode_column	Maximum number of columns of modules. Range: 0 to 30 with the default value of 0 specifying Auto Match.
t	Data compression. 0 - Auto encoding

rotation 1 - Binary mode
 Select rotation.
 0 - no rotation
 1 - rotate 90 degrees
 2 - rotate 180 degrees
 3 - rotate 270 degrees
data The PDF417 barcode data.

Return Value

True -> OK
error_string -> Error Info

Example

datas = 'POSTEKINFO'

```
PTK_DrawBar2D_Pdf417(  
    x_coordinate = 10,  
    y_coordinate = 5,  
    correction_level = 0,  
    px = 3,  
    py = 9,  
    encode_row = 10,  
    encode_column = 3,  
    t = 1,  
    data = datas,  
    rotation = 0)
```

```
PTK_DrawText(  
    x_coordinate=10,  
    y_coordinate=20,  
    fonts = '7',  
    font_size = 4,  
    data='px=3')
```

```
PTK_DrawText(  
    x_coordinate=10,  
    y_coordinate=25,  
    fonts = '7',  
    font_size = 4,  
    data='py=9')
```

```
PTK_DrawBar2D_Pdf417(  
    x_coordinate = 50,  
    y_coordinate = 5,  
    correction_level = 0,  
    px = 6,  
    py = 18,  
    encode_row = 10,  
    encode_column = 3,  
    t = 1,
```

```
data = datas,  
rotation = 0)
```

```
PTK_DrawText(  
    x_coordinate=50,  
    y_coordinate=20,  
    fonts = '7',  
    font_size = 4,  
    data='px=6')
```

```
PTK_DrawText(  
    x_coordinate=50,  
    y_coordinate=25,  
    fonts = '7',  
    font_size = 4,  
    data='py=20')
```

```
PTK_DrawBar2D_Pdf417(  
    x_coordinate = 10,  
    y_coordinate = 40,  
    correction_level = 0,  
    px = 5,  
    py = 10,  
    encode_row = 10,  
    encode_column = 3,  
    t = 0,  
    data = datas,  
    rotation = 0)
```

```
PTK_DrawText(  
    x_coordinate=10,  
    y_coordinate=45,  
    fonts = '7',  
    font_size = 4,  
    data='t=0')
```

```
PTK_DrawBar2D_Pdf417(  
    x_coordinate = 10,  
    y_coordinate = 55,  
    correction_level = 0,  
    px = 5,  
    py = 10,  
    encode_row = 10,  
    encode_column = 3,  
    t = 1,  
    data = datas,  
    rotation = 0)
```

```
PTK_DrawText(  
    x_coordinate=10,  
    y_coordinate=60,  
    fonts = '7',
```



```
font_size = 4,  
data='t=1')
```

```
PTK_PrintLabel(number_of_label=1, number_of_copy=1)
```

Printout



```
px=3  
py=9
```



```
px=6  
py=18
```



```
t=0
```



```
t=1
```

Note: After setting this function, you need to call PTK_PrintLabel to start printing.

3.8.4 PTK_DrawBar2D_DATAMATRIX

Description

Print DATAMATRIX Barcode

Function Definition

```
def PTK_DrawBar2D_DATAMATRIX(  
    x_coordinate: float,  
    y_coordinate: float,  
    multiplier:int,  
    data:string,  
    rotation= 0):
```

Parameters

x_coordinate	The horizontal coordinate of the start point of the barcode. The unit can be set by PTK_SetUnit. Default: mm.
y_coordinate	The horizontal coordinate of the start point of the barcode. The unit can be set by PTK_SetUnit. Default: mm.

rotation Select rotation.
 0 - no rotation
 1 - rotate 90 degrees
 2 - rotate 180 degrees
 3 - rotate 270 degrees

multiplier Module size. Module size in both directions.
data The DATAMATRIX barcode data.

Return Value

True -> OK
error_string -> Error Info

Example

```
PTK_DrawBar2D_DATAMATRIX(  
    x_coordinate = 20,  
    y_coordinate = 220,  
    multiplier = 5,  
    data = " Data Matrix",  
    rotation = 0)  
PTK_PrintLabel(number_of_label=1, number_of_copy=1)
```

Printout

Note: After setting this function, you need to call PTK_PrintLabel to start printing.

3.9 Print 1DBarcode

3.9.1 PTK_DrawBarcode

Description

Print 1D barcodes

Function Definition

```
def PTK_DrawBarcode(x_coordinate: float,
    y_coordinate: float,
    barcode_type: string,
    barcode_height: int,
    wide_unit_width: int,
    data: string,
    human_readable= TEXT,
    narrow_unit_width = 10,
    rotation = 0,):
```

Parameters

x The horizontal coordinate of the start point of the barcode.

y The vertical coordinate of the start point of the barcode.

rotation Select rotation.
 0 - no rotation
 1 - rotate 90 degrees
 2 - rotate 180 degrees
 3 - rotate 270 degrees

barcode Barcode selection.

'barcode' value	Barcode type	Wide bar width
0	Code 128 UCC (shipping container code)	NA
1	Code 128 AUTO	NA
1A	Code 128 subset A	NA
1B	Code 128 subset B	NA
1C	Code 128 subset C	NA
1E	UCC/EAN	NA
2	Interleaved 2 of 5	√
2C	Interleaved 2 of 5 with check sum digit	√
2D	Interleaved 2 of 5 with human readable check digit	√
2M	Matrix 2 of 5	√
3	Code 3 of 9	√
3C	Code 3 of 9 with check sum digit	√
3E	Extended Code 3 of 9	√
3F	Extended Code 3 of 9 with checksum digit	√
9	Code93	NA
E30	EAN-13	NA
E32	EAN-13 2 digit add-on	NA
E35	EAN-13 5 digit add-on	NA

'barcode' value	Barcode type	Wide bar width
E80	EAN-8	NA
E82	EAN-8 2 digit add-on	NA
E85	EAN-8 5 digit add-on	NA
K	Codabar	√
P	Postnet	NA
UA0	UPC-A	NA
UA2	UPC-A 2 digit add-on	NA
UA5	UPC-A 5 digit add-on	NA
UE0	UPC-E	NA
UE2	UPC-E 2 digit add-on	NA
UE5	UPC-E 5 digit add-on	NA

narrow_unit_width Set the narrow bar width.

wide_unit_width Set the wide bar width.

Note: It is only applicable to certain barcodes, please refer to the table above for details.

barcode_height Set the barcode height.

human_readable Barcode text printing.

N - do not print

B - print

data To include variables in the barcode data, it is recommended to take advantage of f-strings that come with Python 3.6+. F-strings can provide a fast and easy way to integrate numeric variables, string variables, container variables, date-times, or your own custom objects as long as `__str__` or `__repr__` method is defined for the class. The combined data string cannot exceed a length of 100 characters.

Return Value

True -> OK

error_string -> Error Info

Example

```
listcode = ['K']
datas = '12345'
for i in listcode:
    codetype = i
    PTK_DrawBarcode(x_coordinate=10,
                    y_coordinate=10,
                    barcode_type= codetype,
                    barcode_height=10,
                    wide_unit_width = 0.6,
                    data=datas,
                    human_readable= TEXT,
                    narrow_unit_width = 0.2,
                    rotation = 0,)
    PTK_DrawText(x_coordinate=2,
                 y_coordinate=30,
                 fonts = '7',
                 font_size = 4,
                 data='narrow_unit_width=0.2')
```

```
PTK_DrawText(x_coordinate=2,  
             y_coordinate=35,  
             fonts = '7',  
             font_size = 4,  
             data='wide_unit_width=0.6')  
PTK_DrawBarcode(x_coordinate=50,  
                y_coordinate=10,  
                barcode_type= codetype,  
                barcode_height=10,  
                wide_unit_width = 0.6,  
                data=datas,  
                human_readable= TEXT,  
                narrow_unit_width = 0.3,  
                rotation = 0,)  
PTK_DrawText(x_coordinate=50,  
             y_coordinate=30,  
             fonts = '7',  
             font_size = 4,  
             data='narrow_unit_width=0.3')  
PTK_DrawText(x_coordinate=50,  
             y_coordinate=35,  
             fonts = '7',  
             font_size = 4,  
             data='wide_unit_width=0.6')  
PTK_DrawBarcode(x_coordinate=10,  
                y_coordinate=45,  
                barcode_type= codetype,  
                barcode_height=10,  
                wide_unit_width = 0.6,  
                data=datas,  
                human_readable= TEXT,  
                narrow_unit_width = 0.3,  
                rotation = 0,)  
PTK_DrawText(x_coordinate=2,  
             y_coordinate=60,  
             fonts = '7',  
             font_size = 4,  
             data='wide_unit_width=0.6')  
PTK_DrawText(x_coordinate=2,  
             y_coordinate=65,  
             fonts = '7',  
             font_size = 4,  
             data='narrow_unit_width=0.3')  
PTK_DrawBarcode(x_coordinate=50,  
                y_coordinate=45,  
                barcode_type= codetype,  
                barcode_height=10,  
                wide_unit_width = 0.9,  
                data=datas,  
                human_readable= TEXT,  
                narrow_unit_width = 0.3,  
                rotation = 0,)
```

```
PTK_DrawText(x_coordinate=50,  
             y_coordinate=60,  
             fonts = '7',  
             font_size = 4,  
             data='wide_unit_width=0.9')  
PTK_DrawText(x_coordinate=50,  
             y_coordinate=65,  
             fonts = '7',  
             font_size = 4,  
             data='narrow_unit_width=0.3')  
PTK_PrintLabel(number_of_label=1, number_of_copy=1)
```

Printout



narrow_unit_width=0.2
wide_unit_width=0.6

narrow_unit_width=0.3
wide_unit_width=0.6



wide_unit_width=0.6
narrow_unit_width=0.3

wide_unit_width=0.9
narrow_unit_width=0.3

Note: After setting this function, you need to call PTK_PrintLabel to start printing.

3.10 RFID Label

3.10.1 PTK_GetPrintJobStatus

Description

Retrieve the current status of the printer.

Function Definition

```
def PTK_GetPrintJobStatus():
```

Parameters

None

Return Value

True -> Print Success
False -> Print Failure

Example

```
PTK_SetPortback(port=PORT_SCRIPT)
PTK_PrintLabel(1, 1)
if PTK_GetPrintJobStatus () == True:
    print(' Print Success ')
```

3.10.2 PTK_ReadRFID

Description

Can be used to read the RFID information of the current label.

Function Definition

```
def PTK_ReadRFID(block = TID, comm_mode = PORT_SCRIPT, auto_foward = False, len = 10):
```

Parameters

block	Reading area of the tag, Default: TID area. Accepted values: TID EPC TID_EPC USER TID_USER RESERVED TID_RESERVED
auto_forward	Specifies whether or not to feed to the next label after the reading of the current RFID label is completed. Default: do not feed.
len	Number of bytes of data to write.

Return Value

data -> Read RFID tag data
 -1 -> Read data Failure

Example

```
data = PTK_ReadRFID(block = TID, comm_mode = PORT_SCRIPT, auto_foward = False)
if data != -1:
    print(data)
```

3.10.3 PTK_WriteRFID**Description**

Can be used to encode RFID tags.

Function Definition

```
def PTK_WriteRFID(data_mode, start_addr, len, block, data):
```

Parameters

data_mode	Specifies the format of the data to write into the RFID tag. Accepted values: HEX ASCII EPC_WRITE
start_addr	Specifies the starting word number for writing to the memory bank. It counts the words from the beginning of the memory bank. Each word represents 2 bytes. This parameter indicates the position within the memory bank where the write operation should start, based on word count. <i>Note: When writing into EPC bank, the 'start_addr' must be 2 or greater; while it can be any value equal to or greater than 0 for other memory banks.</i>
len	Number of bytes of data to write.
block	Reading area of the tag. Accepted values: RESERVED_BLOCK EPC_BLOCK USER_BLOCK
data	Specifies the information intended to be written onto the RFID tag. The data can include a combination of fixed characters represented either in alphanumeric text format or as hexadecimal values. These elements can be combined to form the content of the 'data' parameter. The fixed characters are always placed at the first part of the data. To differentiate between the fixed characters and the counters, or variables, double quotation marks ("") are always used to enclose the fixed characters, regardless of whether the characters are in alphanumeric text format or represented by hexadecimal values. It's important to ensure that the total number of characters in the alphanumeric text format data field or the total number of characters represented by the hexadecimal data is an even number. For example:

- If using alphanumeric text format: "ABCD"C0 is valid, where "ABCD" represents the fixed characters, and C0 represents the counter.
- If using hexadecimal format: "1A2B"V00C0 is valid, where "1A2B" represents the fixed hexadecimal characters, V00 represents the variable, and C0 represents the counter

By enclosing the fixed characters in double quotation marks and ensuring an even number of characters in the alphanumeric text format or the hexadecimal data, you maintain the proper structure and encoding of the data onto the RFID tag.

Return Value

True -> OK
error_string -> Error Info

Example

```
PTK_WriteRFID(data_mode = HEX, start_addr = 2, len = 8, block= EPC_BLOCK, data = "3132")
```

3.11 UI Interface Customlization

3.11.1 PTK_UIInit

Description

Initialize the UI customization interface

Function Definition

```
def PTK_UIInit(*params, require_execute_confirmation = True):->controllers:
```

Parameters

params Use in tandem with PTK_UIPage to define page or pages. Pages serve as a scaffold to hold all UI elements.

Return Value

Dict -> a dict of all the UI element controllers, for all pages and all elements on a page.

Example

defining multiple interfaces:

```
controllers = PTK_UIInit(
    PTK_UIPage('page_0',
        PTK_UIButton(Onpressed=testing1, visible=True),
        PTK_UIText(name='page0', visible=True)
    ),
    PTK_UIPage('send',
        PTK_UIButton(Onpressed=testing2, visible=True),
        PTK_UIText(text='page1', visible=True)
    ),
    require_execute_confirmation = True
)
controllers['page_0']['Widget_2'].update('test') #Update Text
```

defining an interface:

```
controllers = PTK_UIInit(
    PTK_UIPage(no
        PTK_UIButton(Onpressed=testing1, visible=True),
        PTK_UIText(name='text1', visible=True),nbvjjjjm,jjjj
    PTK_UIButton(Onpressed=testing2, visible=True)
    ),
    require_execute_confirmation = True
)
controllers['text1'].update('test')    #Update Text
controller["Widget_1"].disable()    #Enable Button
controller["Widget_3"].enable()    #Disable Button
```

3.11.2 PTK_UIButton

Description

Initialize the UI custom interface button widget

Function Definition

```
class PTK_UIButton(Onpressed, title="button", visible=True, enabled=True, name=""):
```

Parameters

Onpressed Callback function when the button is pressed.
title Button title.Default: "button".
name Name of the Button widget that can be used to refer to the correct controller in the dict returned by UI_Init().
enabled Whether the component is interactive.
visible Whether to display this component.

Return Value

controller -> Button widget object

Example

```
PTK_UIInit(  
    PTK_UIPage(  
        PTK_UIButton(Onpressed=testing1, visible=True),  
        PTK_UIText(title='page0', visible=True),  
    )  
)
```

3.11.3 PTK_UIText

Description

Initialize the UI custom interface text widget.

Function Definition

```
class PTK_UIText(title="text", visible=True, name="", enabled=True) :
```

Parameters

title Text title, default value is "text".
visible Whether to show this component.
enabled Whether the component is interactive
name Name of the Text widget that can be used to refer to the correct controller in the dict returned by UI_Init().

Return Value

controller -> text widget object

Example

```

controllers = PTK_UIInit(
    PTK_UIPage('page_0',
    PTK_UIButton(Onpressed=testing1, visible=True),
    PTK_UIText(title='page0', visible=True),
    )
)
controllers['Widget_2'].update('test') #Update Text

```

3.11.4 PTK_UITextBox**Description**

Initialize the UI custom interface with a titled text box widget.

Function Definition

class PTK_UITextBox(value="--", title="textbox", visible=True, name="", enabled=True) :

Parameters

value	Text box value, the default value is "--".
title	Textbox title, default value is "textbox".
visible	Whether to show this component.
enabled	Whether the component is interactive.
name	Name of the TextBox widget that can be used to refer to the correct controller in the dict returned by UI_Init().

Return Value

controller -> input display box widget object

Example

```

controllers = PTK_UIInit(
    PTK_UIPage('page_0',
    PTK_UIButton(Onpressed=testing1, visible=True),
    PTK_UITextBox (title='page0', visible=True),
    )
)
controllers['Widget_2'].update('test') #Update Text

```

3.11.5 PTK_UIInput**Description**

Initialize the UI custom interface input box widget

Function Definition

class PTK_UIInput(Onsubmit,

```

        Onchange = "",
        title="input",
        name = "",
        value="0",
        visible=True,
enabled=True):

```

Parameters

Onsubmit	The callback function when the input box is submitted.
Onchange	The callback function when the input box is changed, it can be left blank.
name	Name of the Input widget that can be used to refer to the correct controller in the dict returned by UI_Init().
title	The title of the input box, the default value is "input".
value	The input box displays the value.
enabled	Whether the component is interactive.
visible	Whether to display this component.

Return Value

controller -> input box widget object

Example

```

def testing1(value):
    print(value)
def testing2(value):
    print(value)

controllers = PTK_UIInit(
    PTK_UIPage('page_0',
        PTK_UIText(title='page0', visible=True),
        PTK_UIInput(Onsubmit=testing1, Onchange=testing2, visible=True),
    )
)
controllers['Widget_1'].update('test') #Update Text

```

3.11.6 PTK_UICheckBox

Description

Initialize the UI custom interface input box widget

Function Definition

```

class PTK_UICheckBox(Onpressed,
    title="checkbox",
name = "",
    value=False,
    visible=True,
enabled=True):

```

Parameters

Onpressed	Callback function when checkbox is checked.
title	Checkbox title, default value is "checkbox".
name	Name of the CheckBox widget that can be used to refer to the correct controller in the dict returned by UI_Init().
value	The checkbox shows the value, default is false.
enabled	Whether the component is interactive.
visible	Whether to display this component.

Return Value

controller -> Checkbox widget object

Example

```

controllers = PTK_UIInit(
    PTK_UIPage('page_0',
    PTK_UIButton(Onpressed=testing1, visible=True),
    PTK_UIText(title='page0', visible=True),
    PTK_UICheckBox(Onpressed=testing1, visible=True),
)
)
controllers['Widget_3'].update(True) #Update Text

```

3.11.7 PTK_UIList**Description**

Initialize the UI custom interface list widget

Function Definition

```

def PTK_UIList(
    Onpressed,
    items,
    title="list",
    name = "",
    items=['Add Custom List Items', '0', '1'],
    value='Add Custom List Items',
    visible=True,
    enabled=True):

```

Parameters

Onpressed	Callback function when the list is selected.
title	List title, default value is "list".
name	Name of the List widget that can be used to refer to the correct controller in the dict returned by UI_Init().
items	List option value.
value	The list shows the values.
enabled	Whether the component is interactive.

visible Whether to display this component.

Return Value

controller -> list widget object

Example

```
controllers = PTK_UIInit(
    PTK_UIPage('page_0',
        PTK_UIButton(Onpressed=testing1, visible=True),
        PTK_UIText(title='page0', visible=True),
        PTK_UIList(Onpressed=testing1, items = ['0','1','2'], visible=True),
    ),
)
controllers['Widget_2'].update('1') #Update Text
```

3.11.8 PTK_UIPage

Description

Initialize the UI custom interface page

Function Definition

```
def PTK_UIPage(*args):
```

Parameters

***args** The first parameter can be a custom page name, the default is 'page_' + the number of pages, followed by the widget json file that received the page.

Return Value

json -> A json file that records all page information

Example

```
PTK_UIInit(
    PTK_UIPage('page_0',
        PTK_UIButton(Onpressed=testing1, visible=True),
        PTK_UIText(title='page0', visible=True),
        PTK_UIList(Onpressed=testing1, items = ['1','2'], visible=True),
    ),
    PTK_UIPage('send',
        PTK_UIButton(Onpressed=testing2, visible=True),
        PTK_UIText(text='page1', visible=True),
    )
)
controller['page_0']['Widget_2'].update('test')
```

3.11.9 PTK_UIChangePage

Description

User callback to switch the current page.

Function Definition

```
def PTK_UIChangePage(pagenum):
```

Parameters

pagenum For the page to switch, you can enter a custom name or page number, starting from 0.

Example

```
def test1(value):
    PTK_ChangePage('page_0')

PTK_UIInit(
    PTK_UIPage('page_0',
        PTK_UIButton(Onpressed=testing1, visible=True),
        PTK_UIText(title='page0', visible=True),
        PTK_UIList(Onpressed=testing1, items = ['1','2'], visible=True),
        PTK_UICheckBox(Onpressed=testing1, items = ['111','222'], visible=True),
    ),
    PTK_UIPage('send',
        PTK_UIButton(Onpressed=testing2, visible=True),
        PTK_UIText(text='page1', visible=True),
    )
)
```

3.11.10 PTK_UpdateAllFormVariables

Description

This function can be used to replace variables in a pre-made label design template file. Template files can be generated using label editing software to remove the need to code the label design. During the template file generation process, the user can leave fields as variables that can be replaced using this function. Please visit the official POSTEK Github page for tutorial videos.

Function Definition

```
def PTK_UpdateAllFormVariables(filename, **kwargs):
```

Parameters

filename Template file name, the template file must be placed in the directory corresponding to the script file to take effect.

kwargs Variable name and variable value, variable usage in the template file should be defined as #OX:variable name#.

Example

command1.txt:

```
N
A261,1293,0,7,75,75,N,"#OX:Input1#"
A1308,1297,0,7,75,75,N,"#OX:Input2#"
W1,12345
```

test.py:

```
data = ['1202','685161']
cmdd = PTK_UpdateAllFormVariables(
    'command1.txt',
    Input1=data[0],
    Input2=data[1],
)
PTK_SendCmdToPrinter(cmdd)
```

4 API Class

4.1.1 PTK_ExternalStorageDevice

Description

Initialize the External Storage Device.

Class Method

list_all_files	List all file path of current storage device.
read_file	Read a file, specify the file path. return the file read operator.
write_file	Write a file, specify the file path and file operation mode. mode:'w' Rewrite Mode/'a': Append write Mode. return the file read operator.

Example

```
storageDeviceList= PTK_GetExternalStorageDeviceList()
storageDeviceList [0].list_all_files()
storageDeviceList [0].read_file(filename = filename)
storageDeviceList [0].write_file(filename = filename, mode='w')
```

